

P1354R0

SG7 Guidelines for Review of Proposals

Published Proposal, 2018-11-08

This version:

<https://wg21.link/p1354r0>

Author:

[Chandler Carruth](#)

Audience:

SG7

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Guidelines for when a WG21 proposal should be reviewed by SG7, the reflection and compile-time programming study group.

Table of Contents

1 Introduction

2 Compile-Time Programming Facilities

3 Reflection Facilities

3.1 Type Traits

§ 1. Introduction

SG7 is the WG21 study group for Reflection and Compile-Time Programming. It is not a study group for library or API design, especially detailed design. WG21 is seeing an increased number of papers

that potentially overlap with SG7’s domain as the core reflection and compile-time programming facilities become both increasingly available and increasingly in demand. We need guidelines for when proposals need SG7’s specific review.

A core principle of these guidelines is to form basic rubrics, even if somewhat vague, that can be used to choose an initial target subgroup of WG21 for proposal review. The group may end up being incorrect after discussion occurs and the fundamentals of the proposal become better understood. That is working as intended. This system is not intended to be perfect, merely to make misclassification reasonably rare.

§ 2. Compile-Time Programming Facilities

First and foremost, SG7 is responsible for things that satisfy *all* of these things:

- It must be a compile-time construct, and
- It must involve programming, i.e. it must involve *logic* and/or *computation*, and
- It must be a facility for these things, not merely an application or usage of them.

These are our high-level principles for whether a proposal requires SG7 review. Some more specific guidelines to help evaluate proposals:

✗ No Review Needed

Library proposals that merely use *existing* `constexpr` facilities.

EXAMPLE 1

Using compile-time policies to select run-time implementations.

✗ No Review Needed

Library proposals that can use `constexpr` facilities to implement simple, convenience transformations of types and values.

EXAMPLE 2

A library metafunction `std::copy_const`.

✓ Review Needed

Library proposals that form building blocks for logic or computation.

EXAMPLE 3

A library metafunction `std::is_any_type_true_v<Ts...>`.

✓ Review Needed

Library proposals that provide code synthesis.

EXAMPLE 4

A library metafunction `std::generate_wrapper<T>::type`.

§ 3. Reflection Facilities

High level principles of what concerns SG7 has with reflection-related proposals:

- It allows introspection of the source, types, APIs, names, or program, or
- It allows synthesis of source, types, APIs, names, values, or programs.

§ 3.1. Type Traits

✗ No Review Needed

A type trait that can be implemented entirely based upon already approved (by SG7) reflection facilities.

EXAMPLE 5

A trait that exposes information about `enums` via the `enum` reflection facilities.

✗ No Review Needed

A type trait that exposes properties of types that are already clearly observable in the behavior of the type within C++ code.

EXAMPLE 6

A trait that exposes strong structural equality.

EXAMPLE 7

A trait testing for whether an `enum` is a scoped `enum`.

✓ Review Needed

A type trait that exposes information about how source code is written in a way that is not clearly observable by C++ code interacting with the type.

EXAMPLE 8

A trait that exposes the *number* of overloads in an overload set.

EXAMPLE 9

A trait that exposes whether a particular constructor is user-provided —despite contributing to observable properties, this aspect is not directly observed. This is a borderline case, but just over the line towards SG7.